

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Bài 03: Ngăn xếp - Hàng đợi

Nguyễn Thị Tâm
nguyenthitam.hus@gmail.com

Ngày 29 tháng 9 năm 2025

Nội dung

- 1 Kiến thức nền tảng
- 2 Danh sách liên kết - Linked List

Nội dung

1 Kiến thức nền tảng

2 Danh sách liên kết - Linked List

Kiểu dữ liệu nguyên thủy

Kiểu dữ liệu

Các kiểu dữ liệu được đặc trưng bởi

- Tập các giá trị
- Cách biểu diễn dữ liệu được sử dụng chung cho tất cả các giá trị
- Tập các phép toán có thể được thực hiện trên tất cả các giá trị này

Ví dụ các kiểu dữ liệu nguyên thủy trong Java

Bảng 1: Các kiểu dữ liệu cơ bản - Primitive data types

Kiểu	Gt. mặc định	Kích thước	Khoảng giá trị
char	0	2 bytes	0 .. 65535 (Unicode)
boolean	false	1 bit	false/true
short	0	2 bytes	-32768 .. 32767
int	0	4 bytes	-2.147.483.648 .. 2.147.483.647
long	0	8 bytes	$-2^{63} + 1 .. 2^{63}$
float	0.0	4 bytes	$+/- 1.4023 \times 10^{-45} .. 3.4028 \times 10^{38}$
double	0.0	8 bytes	$+/- 4.9406 \times 10^{-324} .. 1.7977 \times 10^{308}$

Kiểu dữ liệu trừu tượng

Kiểu dữ liệu trừu tượng (Abstract Data Type - ADT)

Kiểu dữ liệu trừu tượng bao gồm:

- Tập các giá trị
- Tập các phép toán có thể thực hiện được trên tất cả các giá trị này

Rõ ràng không có cách biểu diễn dữ liệu chung cho dữ liệu trừu tượng

Kiểu	Thành phần	Phép toán
Mảng	Các phần tử	khởi tạo, chèn
Danh sách	Các phần tử	chèn, xóa, tìm,..
Đồ thị	Đỉnh, cạnh	duyet, tìm đường,...
Ngăn xếp	Các phần tử	thêm, lấy,...
Hàng đợi	Các phần tử	thêm, lấy,...
Cây	Gốc, lá, cành	duyet, tìm kiếm,....

Giao diện trong Java - Java Interface

- Mỗi đối tượng tương tác với các đối tượng khác thông qua các phương thức công khai
 - Các phương thức này tạo thành **giao diện lớp**
 - Người lập trình không biết phương thức được cài đặt như thế nào, chỉ cần biết đặc tả vào/ra của phương thức => Tính đóng gói (encapsulation) của lập trình hướng đối tượng
- 2 đối tượng thuộc 2 lớp có thể có các hành vi trên giao diện (nguyên mẫu phương thức) giống nhau, nhưng cách thức thực hiện hành vi (cài đặt phương thức) khác nhau.

Giao diện trong Java - Java Interface

- Các đối tượng thuộc nhiều lớp khác nhau có thể chia sẻ 1 kiểu giao diện (interface) nào đó
 - Các đối tượng này có tất cả các thuộc tính tĩnh và các phương thức khai báo trong giao diện
 - Ngoài các phương thức đã khai báo trong giao diện này, mỗi lớp có thể có các phương thức riêng.
- Kiểu giao diện **interface**
 - Được khai báo trong một tệp có đuôi .java, tên tệp trùng với tên giao diện;
 - Được biên dịch thành tệp mã byte có đuôi .class như các lớp (class) thông thường;
 - Không có hàm dựng; có thể có các thuộc tính tĩnh, chứa một số bất kì các phương thức trừu tượng, tức là phương thức chỉ có khai báo nguyên mẫu, không định nghĩa thân hàm;
 - Từ Java 8, interface còn chứa các phương thức ngầm định **default** và các phương thức tĩnh, các phương thức này có cài đặt thân hàm.

Cách khai báo giao diện

Tên tệp: NameOfInterface.java

```
/* File name: <NameOfInterface>.java */
//import statements
public interface NameOfInterface {
// final, static fields
// method declarations
}
```

Giao diện trong Java - Java Interface

```
public interface Shape {  
    double getPerimeter();  
    double getArea();  
  
    default int compareTo(Shape shape){  
        if(getArea() > shape.getArea()) {  
            return 1;  
        }  
        if(getArea() == shape.getArea()) {  
            return 0;  
        }  
        return -1;  
    }  
}
```

Giao diện trong Java - Java Interface

- Giao diện là kiểu trừu tượng, không thể tạo đối tượng giao diện.

```
Shape shape = new Shape (); // Sai  
Shape shape = new Square(); // Đúng
```

- Lớp sử dụng (cài đặt) giao diện:
 - Một lớp được khai báo là cài đặt một giao diện bắt buộc phải cài đặt (implements) cụ thể tất cả các phương thức trừu tượng của giao diện. Việc cài đặt này được coi là ghi đè (override) lên các phương thức của giao diện.
 - Với các phương thức ngầm định (default), lớp cài đặt giao diện cũng có thể ghi đè lên phương thức này.
 - Lớp cài đặt giao diện có thể định nghĩa phương thức khác có chung nguyên mẫu với phương thức tĩnh của giao diện - đây không phải là thao tác ghi đè. Việc gọi phương thức tĩnh được thực hiện thông qua tên lớp hoặc tên giao diện.
 - Chương trình có thể dùng giao diện làm tên kiểu cho 1 biến đối tượng. Có thể ép kiểu giao diện về kiểu lớp thực tế của biến.
 - Một lớp có thể cài đặt nhiều giao diện.

Giao diện trong Java - Java Interface

Lớp cài đặt một hoặc nhiều giao diện

```
public class NameOfClass implements NameOfInterface1,  
    NameOfInterface2 {  
    // Method implementation of Interface 1  
    // Method implementation of Interface 2  
    // other stuffs (fields, methods) of this class  
}
```

Giao diện lớp

Ví dụ

```
public class Square implements Shape{
    private double length; // field
    public Square(){}
    public Square(double a){
        length = a;
    }
    public double getLength(){
        return length;
    }
    // Implements methods of Shape interface
    public double getPerimeter(){
        return 4*length;
    }
    public double getArea(){
        return length*length;
    }
}
```

Nội dung

1 Kiến thức nền tảng

2 Danh sách liên kết - Linked List

Danh sách liên kết

- Một danh sách liên kết là một cấu trúc dữ liệu bao các phần tử (nút), trong đó, mỗi phần tử chứa một liên kết đến phần tử tiếp theo trong danh sách.
- Danh sách liên kết có thể là danh sách liên kết đơn (*singly linked list*), danh sách liên kết đôi (*doubly linked list*) hoặc danh sách liên kết vòng (*circular linked list*)
- Mỗi phần tử gồm hai phần: dữ liệu và một liên kết đến phần tử tiếp theo trong danh sách.
- Đầu của danh sách (*head*) và được duy trì trong một biến riêng biệt.
- Cuối của danh sách được chỉ định bằng giá trị **null**.

Danh sách liên kết đơn

- Mô tả danh sách liên kết đơn

```
class Node {  
    int data;  
    Node next;  
}  
public Node(int data) {  
    this.data = data;  
    this.next = null;  
}  
Node head = null;
```

Danh sách liên kết đơn

- Mô tả danh sách liên kết đơn với kiểu dữ liệu tổng quát

```
class LinkedList<E>{  
    class Node{  
        E data;  
        Node next;  
        public Node(E data){  
            this.data = data;  
            this.next = null;  
        }  
    }  
}
```

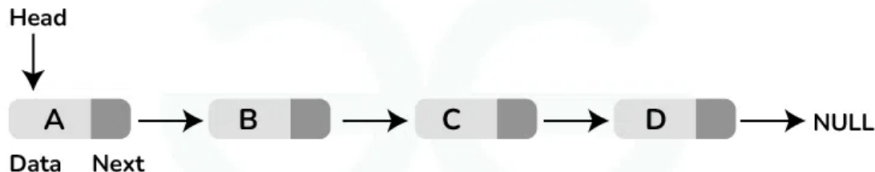
Danh sách liên kết đơn

Lấy độ dài của danh sách

```
Node head;
public int getLength()
{
    int length = 0;
    Node currentNode = head;
    while (currentNode != null){
        length++;
        currentNode = currentNode.next;
    }
    return length;
}
```

Danh sách liên kết đơn

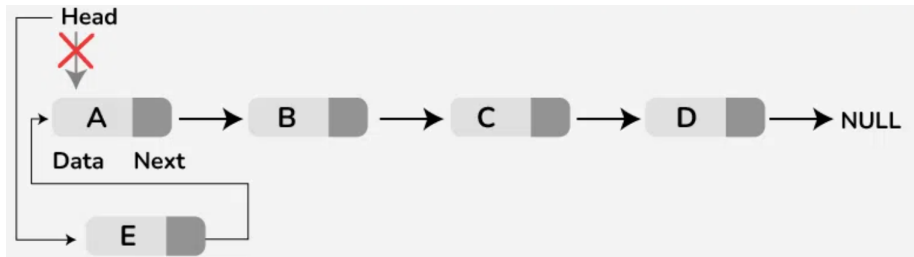
Chèn một phần tử vào đầu danh sách



- Tạo một nút mới: Tạo một nút mới chứa giá trị của phần tử cần chèn.
- Gắn liên kết: Đặt liên kết của nút mới để trỏ đến nút đầu tiên của danh sách hiện tại.
- Cập nhật nút đầu: Gán nút mới làm nút đầu tiên của danh sách.

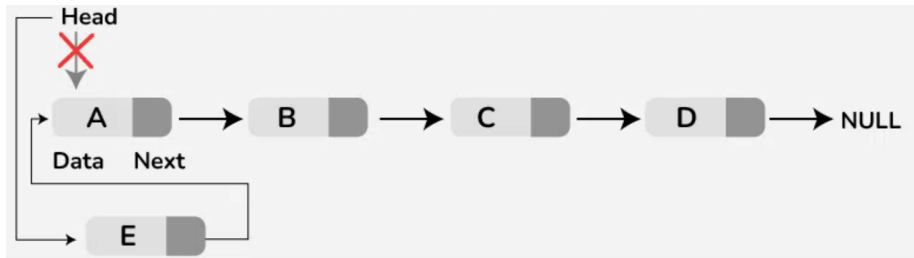
Danh sách liên kết đơn

Chèn một phần tử vào đầu danh sách



Danh sách liên kết đơn

Chèn một phần tử vào đầu danh sách

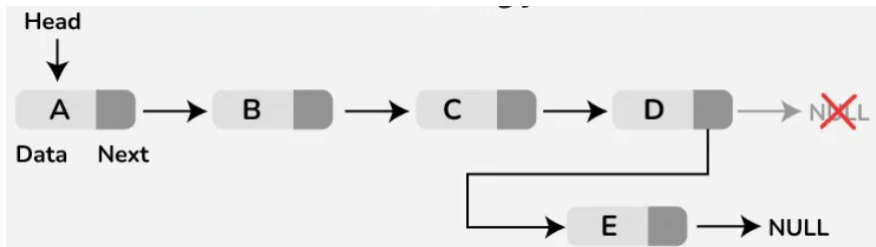


```
public void insertAtBeginning(E data) {  
    Node newNode = new Node(data);  
    newNode.next = head;  
    head = newNode;  
}
```

Danh sách liên kết đơn

Chèn phần tử vào cuối danh sách

- Tạo một nút mới: Tạo một nút mới chứa giá trị của phần tử cần chèn.
- Di chuyển đến cuối danh sách: Lặp qua danh sách cho đến khi đến được nút cuối cùng.
- Gắn liên kết: Đặt liên kết của nút cuối cùng của danh sách để trở đến nút mới.



Danh sách liên kết đơn

Chèn phần tử vào cuối danh sách

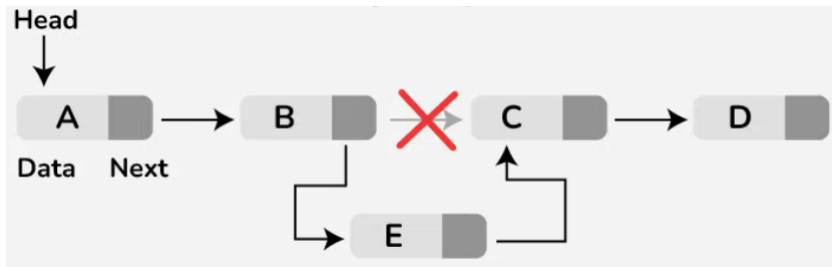
- Tạo một nút mới: Tạo một nút mới chứa giá trị của phần tử cần chèn.
- Di chuyển đến cuối danh sách: Lặp qua danh sách cho đến khi đến được nút cuối cùng.
- Gắn liên kết: Đặt liên kết của nút cuối cùng của danh sách để trỏ đến nút mới.

```
public void insertAtEnd(E data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
    } else {  
        Node current = head;  
        while (current.next != null) {  
            current = current.next;  
        }  
        current.next = newNode;  
    }  
}
```

Danh sách liên kết đơn

Chèn phần tử vào vị trí bất kỳ

- Tạo một nút mới: Tạo một nút mới chứa giá trị của phần tử cần chèn.
- Tìm vị trí chèn: Di chuyển qua danh sách để đến vị trí trước vị trí cần chèn.
- Gắn liên kết: Đặt liên kết của nút mới để trở đến nút tiếp theo của nút hiện tại và gắn liên kết của nút hiện tại để trở đến nút mới.



Danh sách liên kết đơn

Chèn phần tử vào vị trí bất kỳ

```
public void insertAt(E data, int position) {  
    Node newNode = new Node(data);  
    if (position >= 0 and position <= countElement())  
    {  
        if (position == 0)  
            insertAtBeginning(data);  
        else  
            if (position == getLength())  
                insertAtEnd(data);  
            else{  
                Node current = head;  
                int count = 0;  
                while (current != null && count < position - 1) {  
                    current = current.next;  
                    count++; }  
                newNode.next = current.next;  
                current.next = newNode;  
            }  
    }  
}
```

Danh sách liên kết đơn

Chèn phần tử vào vị trí bất kỳ

```
else
{
    try {
        throw new Exception("Invalid position");
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
```

Danh sách liên kết đơn

Xoá phần tử ở vị trí đầu tiên

- Kiểm tra xem danh sách có rỗng không. Nếu danh sách rỗng (không có phần tử nào), không có gì để xóa, chỉ cần trả về.
- Đặt nút đầu tiên của danh sách (head) để trỏ đến phần tử tiếp theo của nó (head.next).
- Giải phóng bộ nhớ của nút cũ (nút bị xóa).

```
public void deleteFirst() {  
    if (head == null) {  
        System.out.println("LinkedList is null");  
        return;  
    }  
    head = head.next;  
}
```

Danh sách liên kết đơn

Xoá phần tử ở vị trí cuối cùng

- Kiểm tra xem danh sách có rỗng không. Nếu danh sách rỗng (không có phần tử nào), không có gì để xóa, chỉ cần trả về.
- Nếu danh sách chỉ có một phần tử, xóa phần tử đó bằng cách đặt head trở về null.
- Di chuyển đến phần tử cuối cùng của danh sách và phần tử trước nó.
- Đặt liên kết của phần tử trước phần tử cuối cùng để trở về null, giải phóng phần tử cuối cùng.

Danh sách liên kết đơn (8)

Xoá phần tử ở vị trí cuối cùng

```
public void deleteLast() {  
    if (head == null) {  
        System.out.println("Linked List is null");  
        return;  
    }  
    if (head.next == null) {  
        head = null;  
        return;  
    }  
  
    Node previous = null;  
    Node current = head;  
    while (current.next != null) {  
        previous = current;  
        current = current.next;  
    }  
    previous.next = null;  
}
```

Danh sách liên kết đơn

Xoá phần tử ở vị trí bất kỳ

- Kiểm tra xem danh sách có rỗng không. Nếu danh sách rỗng (không có phần tử nào) hoặc vị trí cần xóa là vị trí đầu tiên, chỉ cần sử dụng phương thức deleteFirst() để xóa phần tử đầu tiên.
- Di chuyển qua danh sách để đến vị trí trước vị trí cần xóa.
- Xác định phần tử cần xóa và phần tử trước nó.
- Đặt liên kết của phần tử trước phần tử cần xóa để trỏ đến phần tử sau phần tử cần xóa.
- Giải phóng bộ nhớ của phần tử cần xóa.

Danh sách liên kết đơn

Xoá phần tử ở vị trí bất kỳ

```
public void deleteAt(int position) {  
    if (head == null || position == 0) {  
        deleteFirst();  
        return;  
    }  
    Node previous = null;  
    Node current = head;  
    int count = 0;  
    while (current != null && count < position) {  
        previous = current;  
        current = current.next;  
        count++;  
    }  
    if (current == null) {  
        System.out.println("Position is invalid");  
        return;  
    }  
    previous.next = current.next; current = null;  
}
```

Danh sách liên kết đơn (11)

Một số bài tập

- ❶ Viết chương trình để chèn một phần tử vào danh sách liên kết đã được sắp
- ❷ Viết chương trình để đảo ngược một danh sách liên kết
- ❸ Viết chương trình để tìm phần tử lớn nhất/nhỏ nhất trong danh sách liên kết
- ❹ Cho hai danh sách được sắp, viết chương trình để gộp 2 danh sách được sắp thành một danh sách được sắp
- ❺

Danh sách liên kết đơn (11)

So sánh danh sách liên kết đơn và mảng

	Danh sách liên kết đơn	Mảng
Kích thước	Thay đổi	Cố định
Indexing	$O(n)$	$O(1)$
Chèn tại vị trí đầu	$O(1)$	$O(n)$
Xoá tại vị trí đầu	$O(1)$	$O(n)$
Chèn tại vị trí cuối	$O(n)$	$O(1)$
Xoá tại vị trí cuối	$O(n)$	$O(1)$